

Mobile Application Development

CTEC3703

Module Team: Dr Ali Jwaid & Dr Usama Mannai & Dr Taha Rassem

2024/25

1.) Analysis and design:	2
1.A) functional Requirements	2
1.B) Use Cases descriptions and diagrams	4
USD – 1 : User logs in	5
USD – 2: User Creates account.....	5
1.C) UML design or Class diagrams	6
UML Diagram 1	6
Class Diagram 1	7
1.D) User Interface design	8
2.) Database	10
2.A) Database Tables.....	10
2.B) ERDs	12
ERD created at 28/10/24.....	12
3.) Software Architectual patterns.....	12
UI Logic	13
Input Logic.....	15
Create Account page.....	15
Login Page	16
Quantity Page	17
Business Logic	18
PutExtra Ability.....	18
Databases	19
4.) Testing and Security Consideration	20

1.) Analysis and design:

1.A) functional Requirements

- Register and Log in securely to access their account
 - A user will have to use valid login details (a username/email and password) to access the rest of the app
 - The User will be able to create an account, and then use these details to log in.
- Browse the cafe's menu, including drinks and cakes. And Select items to add to their order.
 - The entire cafes stock will be presented to the user once logged in successfully. And will be available to add to their order list.
 - They will be able to also choose the quantity of the item to add to order
- View and modify their order in a cart.
 - They will be able to view and edit their personal 'order' list of all things they have selected from the Cafes stock
 -
- Place the order and receive a confirmation.

-They can confirm their 'order' list, and it will be processed. It will add the order to a queue to be delivered to the customer who ordered it. It will also send a confirmation email/text message to the customer who ordered it. Making it clear that the order has gone through.
What was ordered and when it will arrive

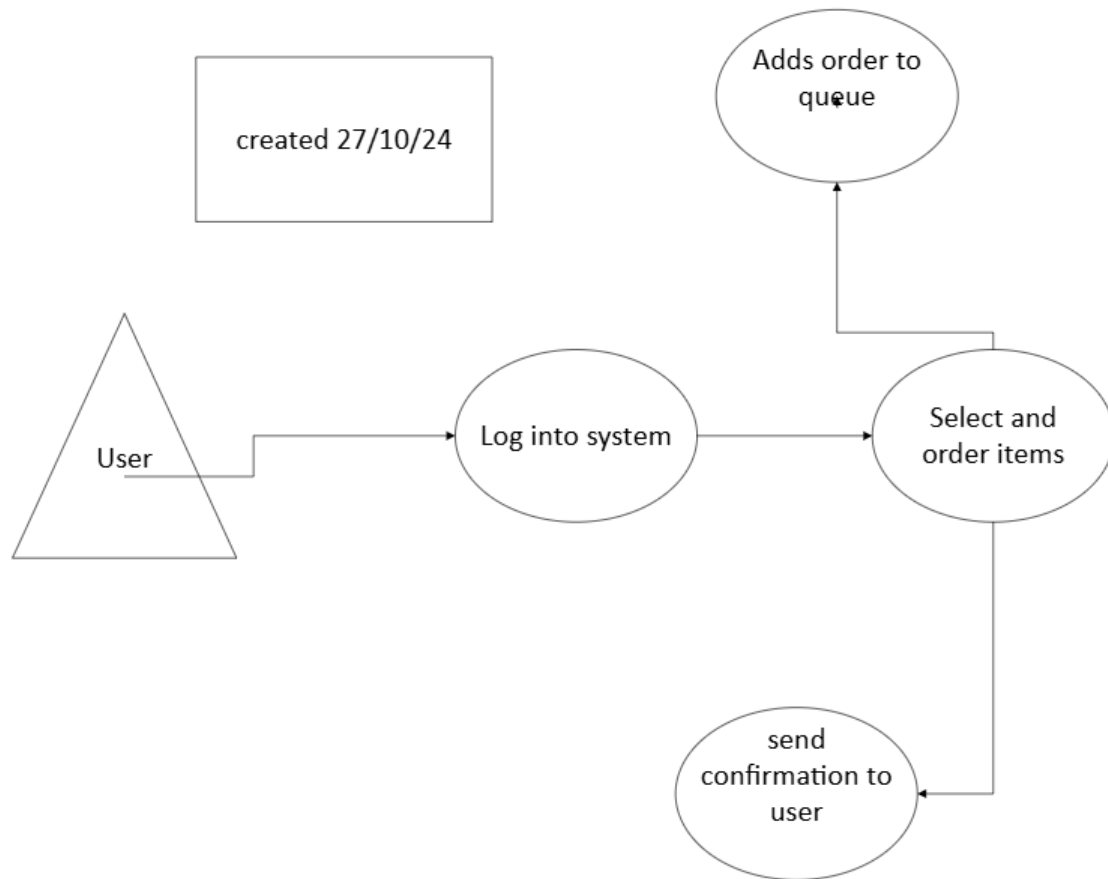
- Write feedback and ratings.

The Customer will have the opportunity to write any feedback notes or opinions they would like to be reviewed by the café business. Also, an optional 5- star rating feature will be presented

- Receive notifications on order status updates.

As the order is processed, the customer will receive updates. Such as order received, order being prepared, quality check, out for delivery, delivered etc...

1.B) Use Cases descriptions and diagrams



USD – 1 : User logs in

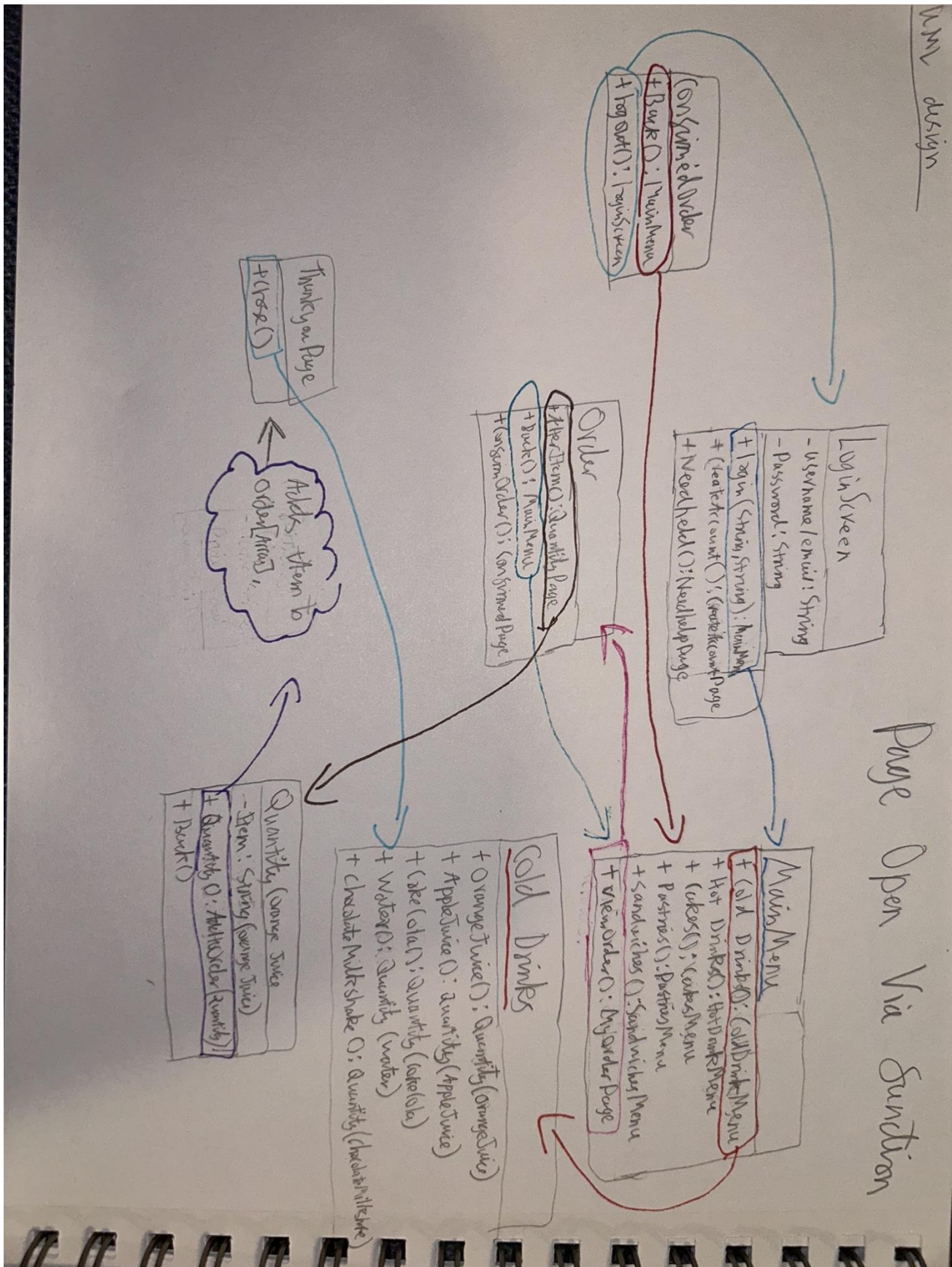
Description	The Customer can log into the app with a valid login detail
Actors	The Customer
Preconditions	The Customer has an account already existing They are using existing login details that are stored in the system
Postconditions	The user gains access to the app.
Trigger	The App allows the user to move into selecting items on the list.

USD – 2: User Creates account

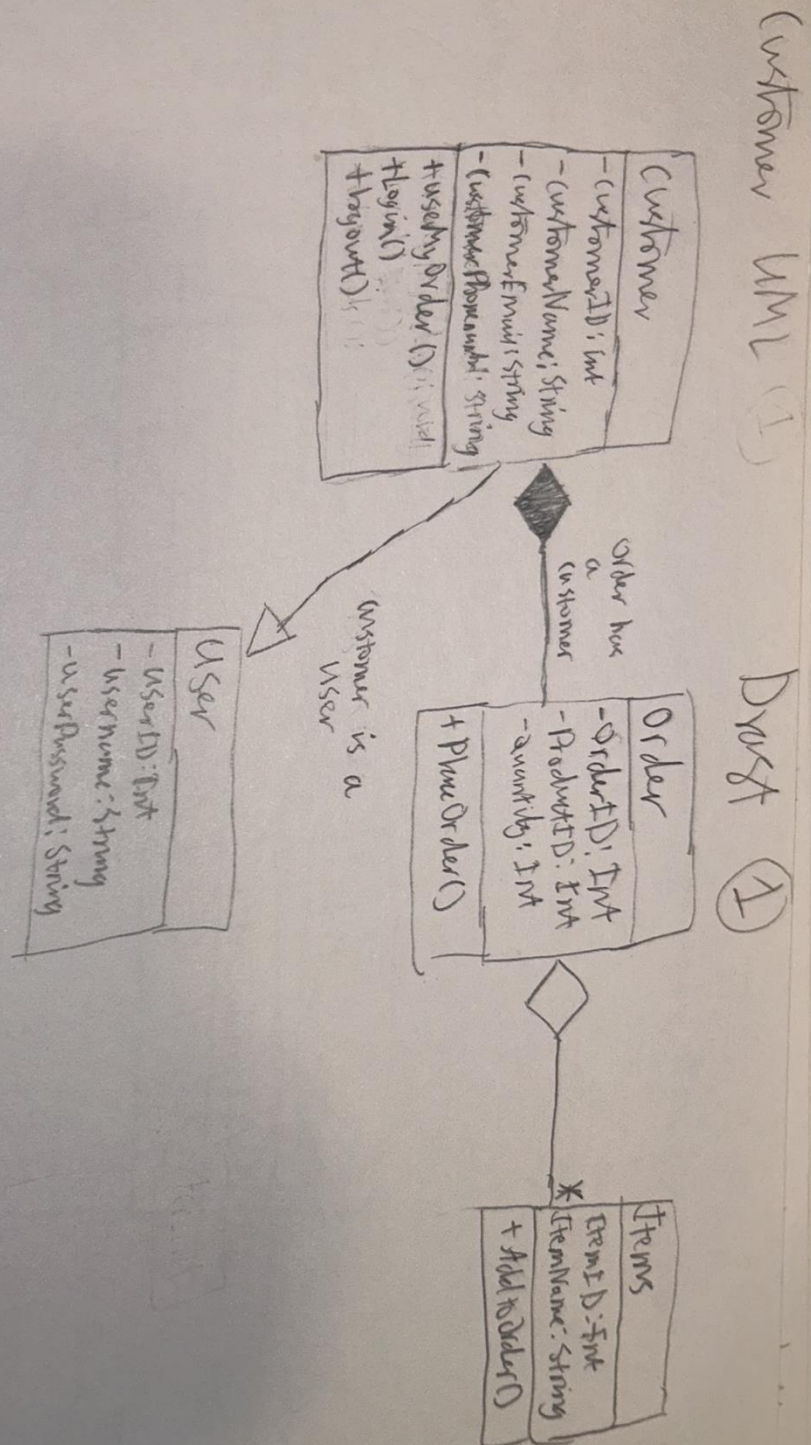
Description	The user creates an account on the app.
Actors	The Customer
Preconditions	The Customer does not have an account, and they use account details that are not already stored in the system
Postconditions	The user has an account created
Trigger	The App stores the new account

1.C) UML design or Class diagrams

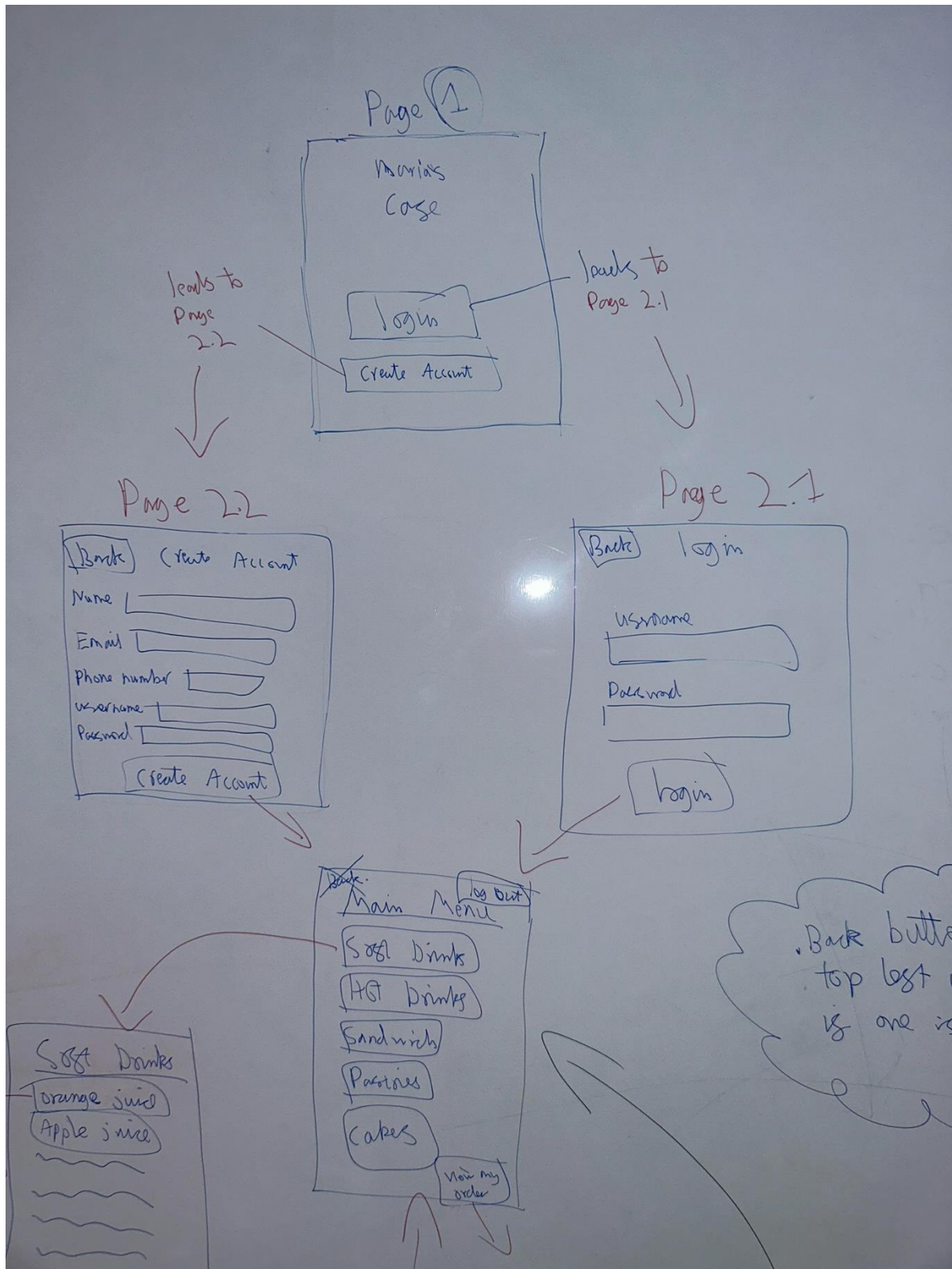
UML Diagram 1



Class Diagram 1



1.D) User Interface design



password:

Main Menu

Back top is

Soft Drinks


~~~~~  
~~~~~  
~~~~~  
~~~~~  
~~~~~

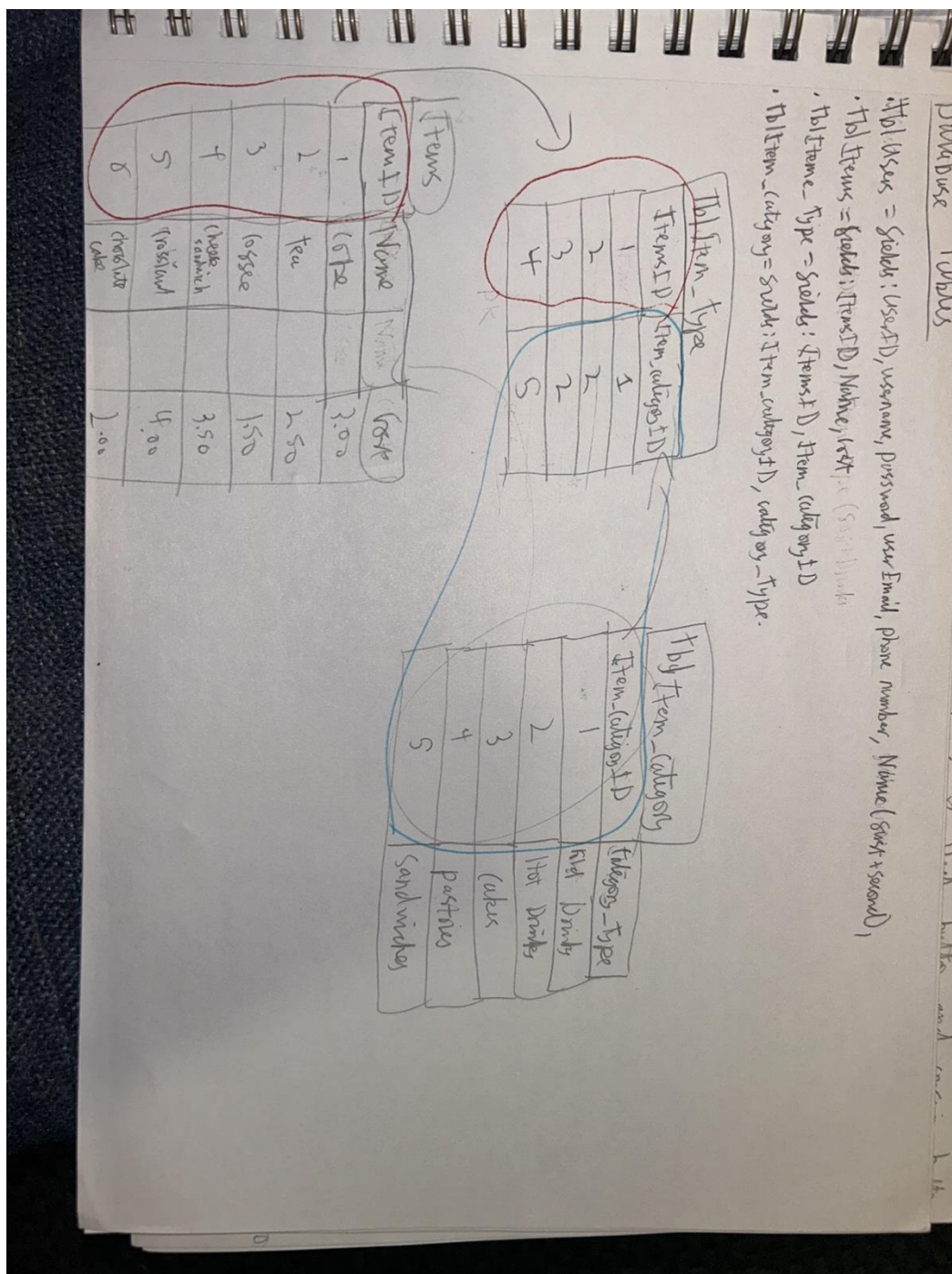
Orange Juice  
Image Price  
Amount Price  
Quantity

My order  
1x ~~~~~   
2x ~~~~~

Thank you  
for ordering  
At Marinos  
Cafe! A  
confirmation  
will be sent  
shortly

## 2.) Database

### 2.A) Database Tables



TblUser – holds account details.

| UserID<br>(Int) | FirstName<br>(String) | LastName<br>(String) | Email<br>(String) | Phone<br>Number<br>(Int Long) | Address<br>(String) | Username<br>(String) | Password<br>(String) |
|-----------------|-----------------------|----------------------|-------------------|-------------------------------|---------------------|----------------------|----------------------|
|                 |                       |                      |                   |                               |                     |                      |                      |

TblItems – Holds the details of all the items sold on this app.

| ItemsID ( Int ) | Name ( String ) | Category ( Int ) | Cost ( Double ) |
|-----------------|-----------------|------------------|-----------------|
|                 |                 | ( FOREIGN KEY )  |                 |

TblItemCategories – Holds the Categories of Items

| ItemCategoriesID (Int) | ItemType (String) |
|------------------------|-------------------|
| ( PRIMARY KEY )        |                   |

The reason for TblItemCategories is an attempt of normalisation.

Was having the issue with the table being created the first time the program was ran , then errors would pop up for when it tried to create and populate the table again.

2024-11-07 21:49:50.020 4051-4051 SQLiteLog com.example.mariascafe E (1) table TblUser already exists in "CREATE TABLE TblUser ( UserID INTEGER PRIMARY KEY AUTOINCREMENT, FirstName TEXT NOT NULL, LastName TEXT NOT NULL, Email Text NOT NULL,Phonenumber TEXT NOT NULL, Address TEXT,

2024-11-07 21:49:50.037 4051-4051 SQLiteDatabase com.example.mariascafe E Error inserting LastName=McCabe UserName=Moo Email=morgan.mccabe@email.com Address=8 Harrier close Password=Password Phonenumber=07927407849 FirstName =Morgan

However, when creating the table ‘TblOrder’, this table had to be empty every time the program was booted up. The most effective way for this was at the beginning of the program, check if table exists. If so, delete it and recreate it.

This method in the OrderDBHelper

```
fun CreateTable() {  
    val db: SQLiteDatabase = this.writableDatabase  
    db?.execSQL("DROP TABLE IF EXISTS " + TableName) // deletes table  
    try { // MASSIVE ISSUE. GAP BETWEEN THE " and + !!!!!  
        val sqlCreateStatement: String = "CREATE TABLE " + TableName + " (" +  
Column_ID +  
        " INTEGER PRIMARY KEY AUTOINCREMENT, " + Column_ItemName + " TEXT NOT  
NULL, " +  
        Column_Quantity + " INT NOT NULL, " + Column_Cost + " DOUBLE NOT NULL  
)"  
        db?.execSQL(sqlCreateStatement) // creates table  
    } catch (e: SQLiteException) {  
    }  
}
```

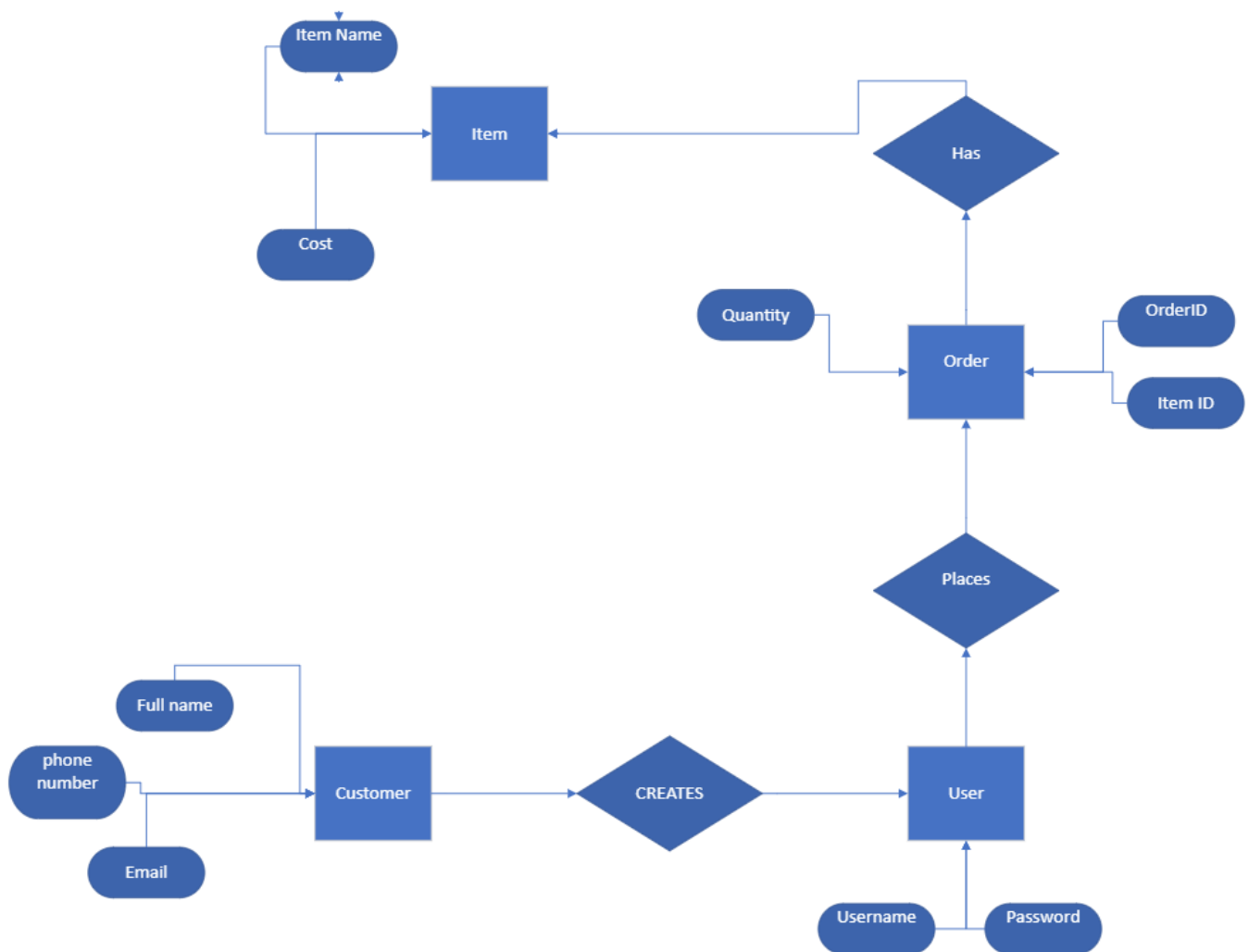
I then implemented a similar tactic with all other tables. It is crucial for the others much more since they have data auto inserted at the time they are created. This removes all errors popping up.

An interesting find was you cannot put the ‘DROP TABLE’ statement in the Oncreate() method. It does not seems to work. It had to be put in an entirely new method, and that method had to be called .

I made a decision early on that all tables will be initialised in the StartMenu class in the Oncreate() method. As soon as the app loads up the tables will be created and populated ( if necessary )

## 2.B) ERDs

ERD created at 28/10/24



## 3. ) Software Architectual patterns

## UI Logic

The First Objective was to have a functional UI where a user can simply navigate from the start of the application to the end. And then back again. For you can see in [UML Diagram 1](#) , this was the first draft of an idea of which class would leave to. It would use a function, that is connected to a button. And when the button is clicked by the user, they will start the next class (activity ).

### EXAMPLE

The method (also known as function) is connected to the Onclick attribute in the button in the page layout.

```
<Button
    android:id="@+id/button2"
    android:layout_width="240dp"
    android:layout_height="95dp"
    android:layout_marginStart="85dp"
    android:layout_marginTop="82dp"
    android:layout_marginEnd="86dp"
    android:onClick="LoginButton"
    android:text="Login"
    app:layout_constraintBottom_toBottomOf="parent"
    app:layout_constraintEnd_toEndOf="parent"
    app:layout_constraintHorizontal_bias="0.0"
    app:layout_constraintStart_toStartOf="parent"
    app:layout_constraintTop_toBottomOf="@+id/textView"
    app:layout_constraintVertical_bias="0.0" />
```

Once the button is clicked by the user, it runs the startActivity method. This runs the class that is declared. In the code underneath it is the class 'LoginPage'

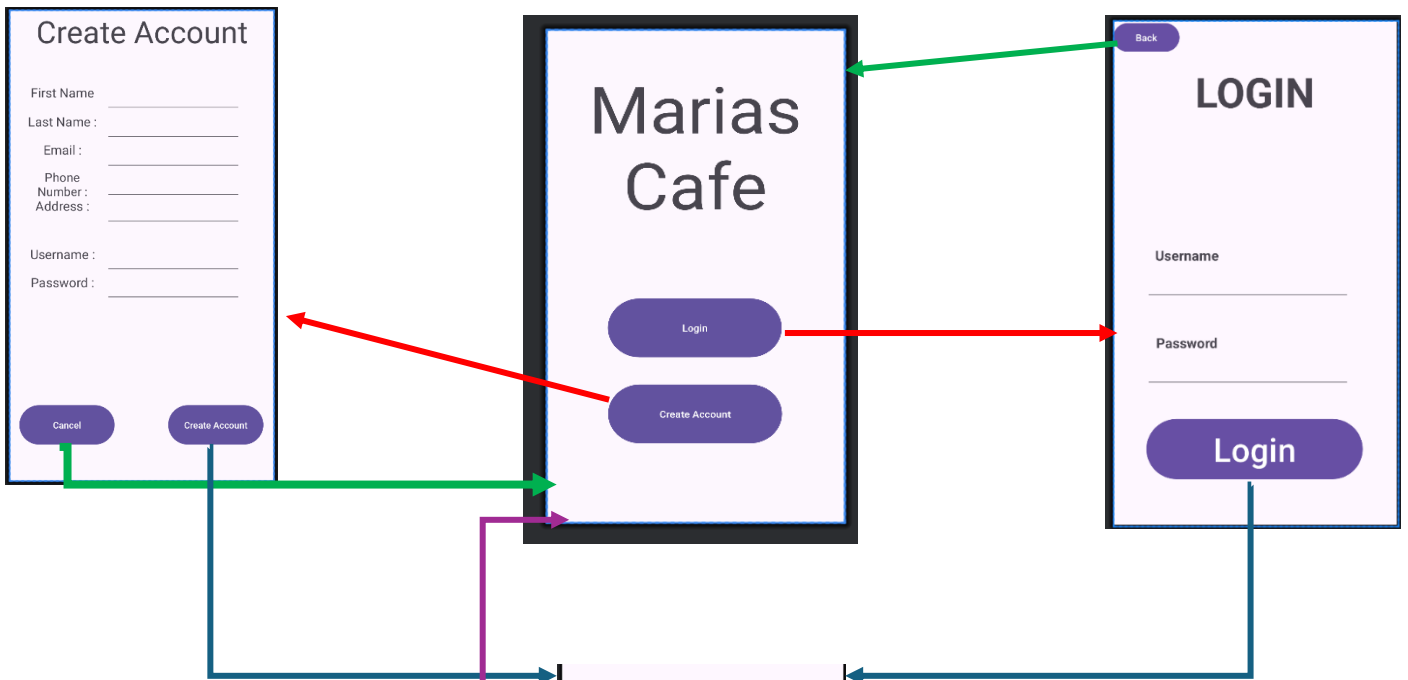
```
val send = Intent(this, LoginPage::class.java)
startActivity(send)
```

This will be the main method on navigating the UI. All Pages should also have a 'Back' button, where it simply navigates back to the page the user just came from. This will be using the exact same attractivity as seen before.

I wanted to create a UI that flowed logically, for example. It would make little sense that as soon as you log in, you are taken to the current order page. Since you haven't had the time or chance to add anything to your order, you'll be looking at an empty screen. It would be most sensible to take the user to the Main Menu page, where here they can navigate to see and select all of the items that are available to add to their order.

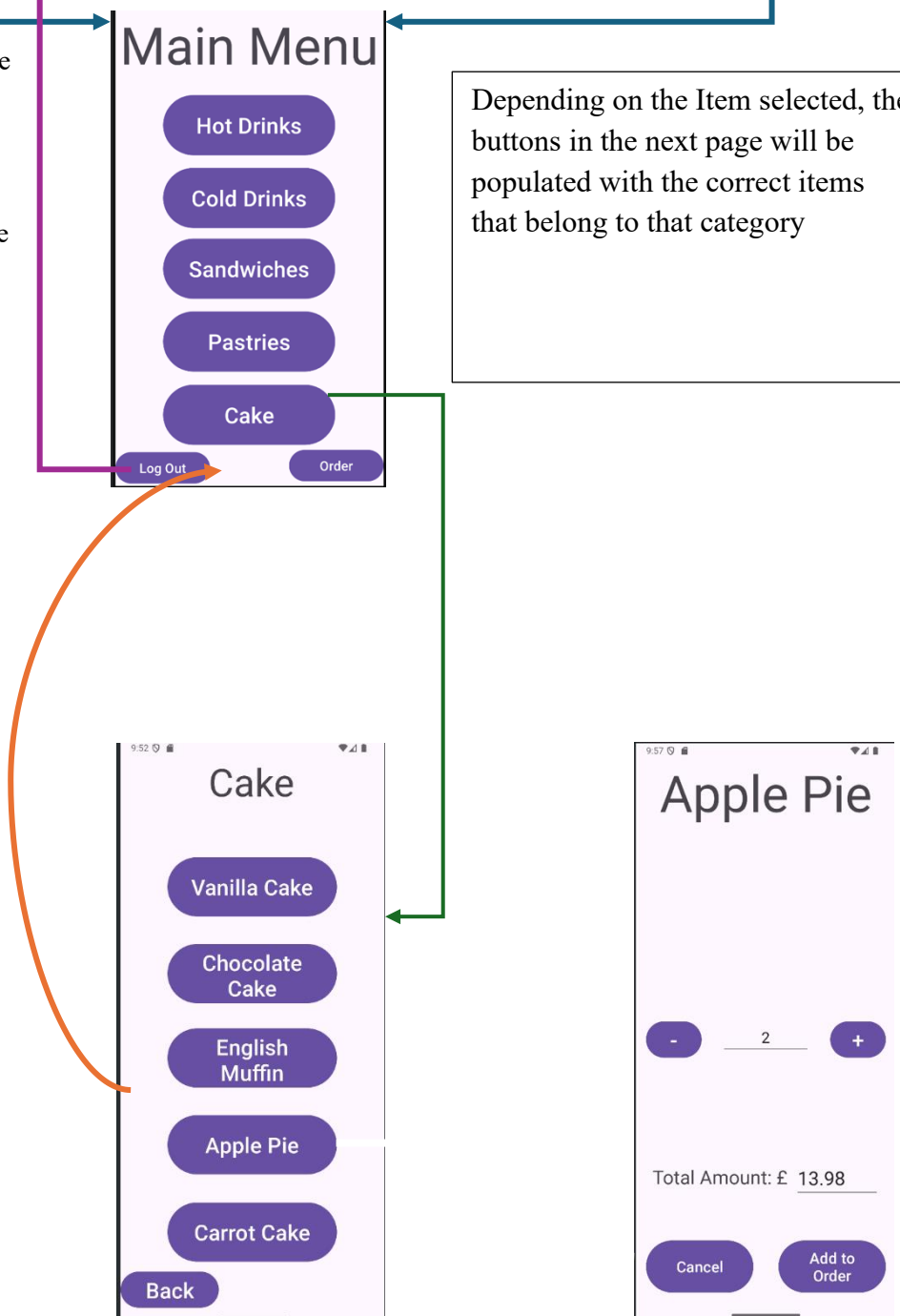
Additionally, I did not want to always make every single page available, for this would

- A.) Decrease security (such as on the log in page, they can access the menu pages. ) Skipping past validation
- B.) Would make the pages feel less structured a tamper with the logical flow of the pages



The original plan was to have 5 separate Classes, one for each specific menu (Pastries, cake etc.. ) but due to the Ability of passing through variables, we Can now just use one class (more on This in the business logic section)

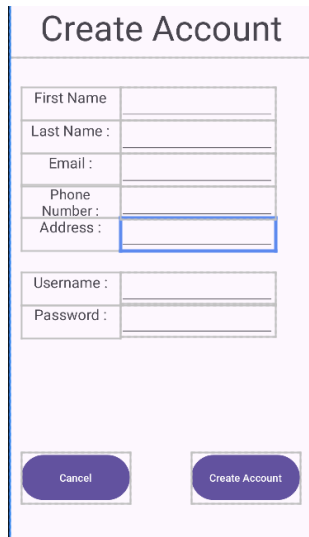
Depending on the Item selected, the buttons in the next page will be populated with the correct items that belong to that category



## Input Logic

All pages have buttons where the user can 'input'. This is one type of inputting from the user. However there are some pages with more intricate input methods. Such as :

### Create Account page



In this page it accepts 7 values. Once entered, the 'Create Account' button saves this data into the TblUser Table into a row. The Username and password can then be used to log in in the 'Log in' page. The boxes next to the texts are editable, but the boxes with text in them are not editable.

All are accepted as String, except for the 'Phone Number' field.

Collects the data inputted from the Create Account Page

This method is from the CreateAccount class,

```
fun CreateAccount(view: View){  
  
    val TblUser = UserDBHelper( context: this)  
  
    val FirstName = findViewById<EditText>(R.id.editTextTextFirstname).text.toString()  
    val LastName = findViewById<EditText>(R.id.editTextTextLastname).text.toString()  
    val Email = findViewById<EditText>(R.id.editTextTextEmailAddress).text.toString()  
    val PhoneNumber = findViewById<EditText>(R.id.editTextPhone).text.toString()  
    val Address = findViewById<EditText>(R.id.editTextTextAddress).text.toString()  
    val Username = findViewById<EditText>(R.id.editTextTextUsername).text.toString()  
    val Password = findViewById<EditText>(R.id.editTextTextPassword).text.toString()  
  
    val new_user = User(Username,FirstName,LastName,Address,Password,Email,PhoneNumber)  
    TblUser.addUser(new_user)  
  
    val send = Intent( packageContext: this, MainMenu::class.java)  
    startActivity(send)  
  
    //addUser  
}
```

Passes the data as object 'User' to UserDBhelper to insert as a row.

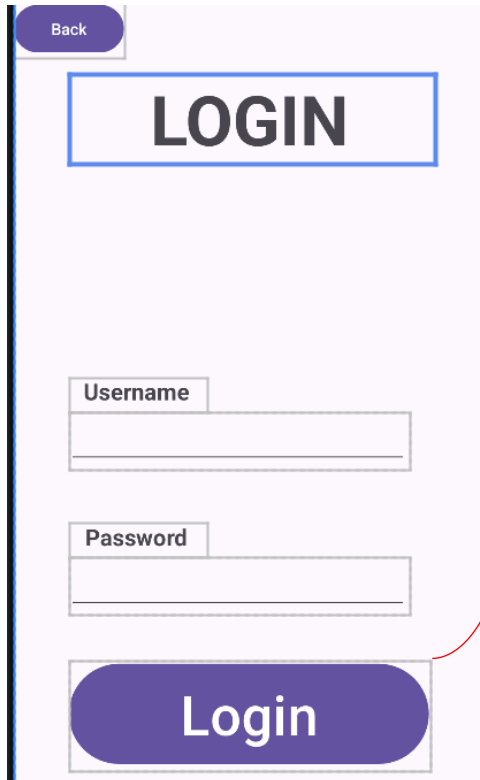
Navigates the User to Main Menu, passing them through login validation stage

```
// ADDING DATA TO USER TABLE  
fun addUser(user: User): Int {  
  
    val db: SQLiteDatabase = this.writableDatabase  
  
    val cv: ContentValues = ContentValues()  
  
    //cv.put(Column_ID, user.UserID)  
    cv.put(Column_FirstName, user.FirstName)  
    cv.put(Column_LastName, user.LastName)  
    cv.put(Column_Email, user.Email)  
    cv.put(Column_Address, user.Address)  
    cv.put(Column_PhoneNumber, user.Ponenumber)  
    cv.put(Column_UserName, user.Username)  
    cv.put(Column_Password, user.Password)  
  
    val success = db.insert(TableName, nullColumnHack: null, cv)  
  
    db.close()  
  
    if (success.toInt() == -1) return success.toInt() //Error, adding new user  
    else return success.toInt() //1  
}
```

Method found in UserDBhelper

This adds the data to the table as a row, allowing this user to now use their details to login.

## Login Page



The image shows a mobile app login screen. At the top left is a 'Back' button. In the center is a large 'LOGIN' title. Below the title are two input fields: 'Username' and 'Password'. At the bottom is a large purple 'Login' button. A red arrow points from the 'Login' button to the first code block, and a green arrow points from the 'checklogindetails' method in the second code block to the 'Login' button.

The User can use their login details to gain access to the apps services. Their Details should already be in TblUser

Method found in LoginPage class, It collects the data inputted, and passes them through method 'checklogindetails' to compare details found in TblUser

```
//login button
fun Login(view: View) {

    val Username = findViewById<EditText>(R.id.editTextUsername).text.toString()
    val Password = findViewById<EditText>(R.id.editTextPassword).text.toString()

    val UserDB = UserDBHelper( context: this)
    val valid = UserDB.checklogindetails(Username, Password)

    // findViewById<TextView>(R.id.txtlogin).text = "yay"

    if(valid == 1){
        val send = Intent( packageContext: this, MainMenu::class.java)
        startActivity(send)
    }
}
```

The method to the right is found in UserDBHelper. It uses the two arguments provided in an SQL statement, then checks if it could find any rows that fit the conditions. If so, the user will be allowed to access the app

```
// checks if the username and password is found in the Users Table
fun checklogindetails(username: String, password: String): Int {

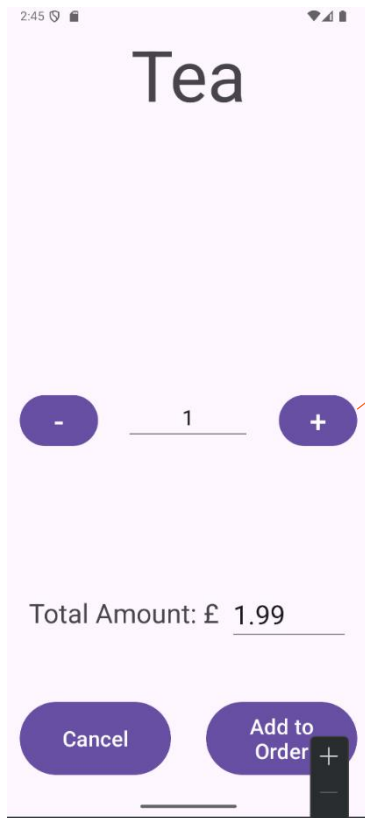
    val db: SQLiteDatabase = this.readableDatabase

    val param = arrayOf(username, password)
    val cursor: Cursor = db.rawQuery(
        sql: "SELECT * FROM $TableName WHERE $Column_UserName = ? AND $Column_Password = ?",
        param
    )

    if (cursor.moveToFirst()) { // false if cursor is empty.
        cursor.close()
        db.close()
        return 1
    }

    cursor.close()
    db.close()
    return 0
}
```

## Quantity Page



Once the user selects an item, they will be able to set an amount ( quantity ). Using the plus and minus button to amend the number in the middle. It is capped so It cannot be less than 1, or more than 99. Then if the user click 'Add to Order' , the item name, quantity and total cost Is saved into the table TblOrder.

```
fun Increase(view: View){  
    var Quantity = findViewById<TextView>(R.id.TxtQuantity).text.toString().toInt() + 1  
    val ItemCost = intent.getSerializableExtra( name: "ItemCost") as Double  
    if(Quantity < 100){  
        val TotalCost = (ItemCost * Quantity)  
        findViewById<TextView>(R.id.TxtQuantity).text = Quantity.toString()  
        findViewById<TextView>(R.id.txtCost).text = TotalCost.toString()  
    }  
}
```

Validates new quantity  
number, and sees if it  
allowed  
  
 $99 > \text{Quantity} > 0$

## Business Logic

### PutExtra Ability

The ability to pass data to the new activity dramatically decreased the number of classes required. For example the MainMenu class to the SelectedMenu class



```
fun SelectMenu(view: View){  
  
    val b = view as Button  
    val buttonText = b.text.toString()  
  
    val send = Intent( packageContext: this, SelectedMenu::class.java).apply{  
        putExtra( name: "MenuTitle", buttonText)}  
        startActivity(send)  
    }  
}
```

All 5 of these buttons onclick attribute is set to this method. The view allows us to declare which button was selected exactly. With this, we extract the Text of the button to pass to the new activity. With this variable we can set the title of the next page and set the next 5 buttons to the correct items. Without this ability, we would need 5 separate methods that start to their own activity / class. It would increase the quantity of code and classes by 5, decreasing the level of optimisation of the program.

In the “Selected Menu” class, the way to initialise the argument passed on, we do:

```
MenuTitle = intent.getSerializableExtra("MenuTitle") as String
```

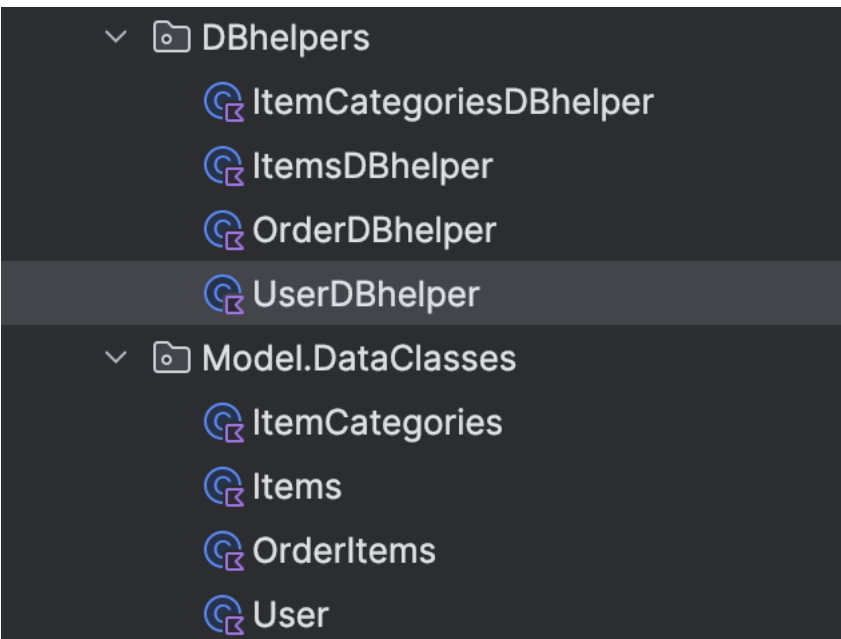
Using this variable ‘MenuTitle’, we can use methods from ItemsDBhelper to fetch the relevant items from TblItems. It creates a List ‘ItemsList’, which is created by Object ‘Items’ which withhold values : Name, Category, Cost.

When a button is selected, the putExtra ability is used again to pass through the data of the selected Item to the Quantity page.

```
fun SelectedItem(view: View) {  
    val b = view as Button  
    val buttonTag = b.getTag().toString().toInt()  
  
    //findViewById<TextView>(R.id.txtMenu).text = buttonText.toString()  
  
    val send = Intent(this, QuantityPage::class.java).apply{  
        putExtra("MenuTitle", MenuTitle)  
        putExtra("ItemName", ItemsList.get(buttonTag).Name)  
        putExtra("ItemCost", ItemsList.get(buttonTag).Cost)  
    }  
    startActivity(send)  
}
```

## Databases

There are 4 SQL tables used. TblUser, TblItems, TblOrders, TblItemCategories. Each have a corresponding Data class



In a MVC structure, this would be the Model section. The reason for the Data classes is to ease the accessibility to inserting and fetching the data from the SQL Tables.

In the Controller section ( the classes that hold the activity methods ) they connect to the databases by:

```
val TblItems = ItemsDBhelper(this)
```

Now you have access to the TblItems table, and can use the methods from ItemsDBhelper on it.

```
TblItems.Populate_ItemTable(Items, Cost)
```

The Data classes hold values that are identical to the columns of the tables. For example,

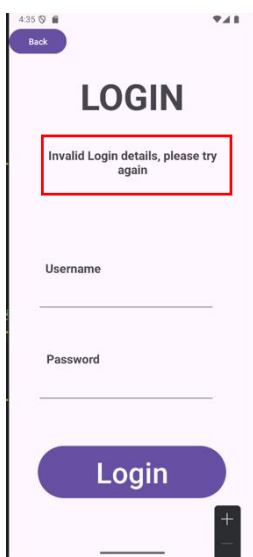
```
data class User(var Username: String, var FirstName: String, var LastName: String,
               var Address: String, var Password: String, var Email: String, var
               Phonenummer: String){}
```

And the TblUsers columns are:

```
private val TableName = "TblUser"
private val Column_ID = "UserID"
private val Column_FirstName = "FirstName "
private val Column_LastName = "LastName"
private val Column_Email = "Email"
private val Column_Address = "Address"
private val Column_PhoneNumber = "Phonenummer"
private val Column_UserName = "UserName"
private val Column_Password = "Password"
```

## 4.) Testing and Security Consideration

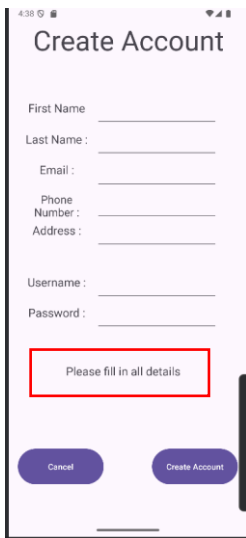
### SECURITY



For security, this will predominantly be focused on the logging in / creating an account section

When logging in, it is crucial that the login details are checked to the ones stored in the database. If they are not found, the user is informed with a message that appears on the page. The user only gains access if the correct username/password is entered.

When it comes to creating an account, the user must fill in all fields to be allowed access / create an account. If there is a field left blank ( even if it is just whitespaces) the user will be warned of this



4:38

### Create Account

First Name \_\_\_\_\_

Last Name : \_\_\_\_\_

Email : \_\_\_\_\_

Phone Number : \_\_\_\_\_

Address : \_\_\_\_\_

Username : \_\_\_\_\_

Password : \_\_\_\_\_

Please fill in all details

Cancel Create Account

## 5.) Considerations

### Pop-up Windows

A major improvement in this app would be pop-up windows, such as when you login, a temporary window from the bottom of the screen appears with the text such as ‘you logged in successfully’. This gives the app a stronger feel of confirmation and professionalism.

Also, it can give the user additional info without having to dedicate a whole page / load of a new activity.

### Initialising variables

I found initialising variables at the beginning of a class to be challenging. Somewhat to do with not knowing the correct syntax, but also I found that initialising the variables in the Oncreate() method would not make them public for all other methods in the class. I believe this is due to the Oncreate() , though a special method , is still a method fundamentally. So, any variable initialised would be only for local use. Therefore, It would need to be initialised before the Oncreate() method.

Example in the SelectedMenu class . the 'MENUTITLE' variable

```
class SelectedMenu : AppCompatActivity() {  
  
    var TblItems: ItemsDBhelper = ItemsDBhelper(this)  
    lateinit var ItemsList: ArrayList<Items>  
    lateinit var MenuTitle:String
```

The Oncreate() method

```
MenuTitle = intent.getSerializableExtra("MenuTitle") as String
```

...

It can now be used in separate methods, such as the SelectedItem method

```
fun SelectedItem(view: View) {  
    val b = view as Button  
    val buttonTag = b.getTag().toString().toInt()  
  
    //findViewById<TextView>(R.id.txtMenu).text = buttonText.toString()  
  
    val send = Intent(this, QuantityPage::class.java).apply{  
        putExtra("MenuTitle", MenuTitle)
```

## UI detail / décor

The ui is functional, however it is bland and uninteresting. This is due to time constraints. But an experimentation of font type and colour, button colours and shape. And hand drawn images for the background would increase the apps user friendliness. Aswell as more info of the items, such as ingredients, where they were made, a picture of the item etc...

## Databases

I would like to implement a higher level of normalisation, where I officially create the foreign and primary key connection between tblitem and tblitemcategories.

Implementing a TblUserOrders would be for saving multiple orders made by a single user